

Laboratory Report:
Artificial Intelligence in Games

Part 1. Bot behavior: optimization vs. learning vs. heuristics

19 września 2026

Instructor: dr hab. inż. Maciej Komosiński, prof. PP

Authors: **Damian Krzyżelewski** 151796

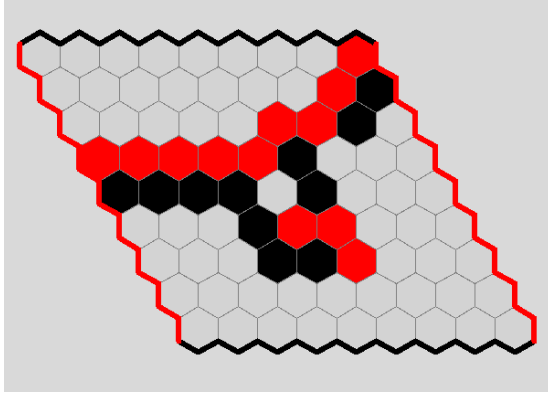
Classes: every other Monday (even weeks), 15:10 - 16:40.

I/We declare that this report and the accompanying source code have been prepared exclusively by the above author(s), and all elements derived from other sources have been properly marked and are cited in the bibliography.

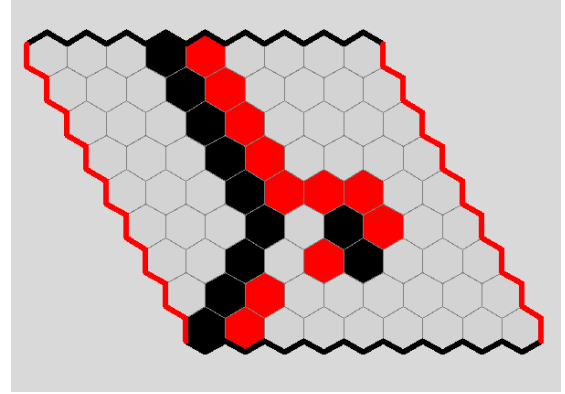
1 Introduction, rules, and basic strategies of the chosen game

The game of **Hex** was chosen. The gameplay takes place on a 9×9 board consisting of hexagonal cells. Hex is a classic zero-sum strategic board game with perfect information and no random elements. Due to topological reasons, there are no draws in the game - every match ends with a victory for one of the sides [2].

The game involves two players, each assigned to two opposite edges of the board (denoted by different colors). A player's goal is to form an unbroken chain of their stones connecting their edges through adjacent cells. Players take turns making moves (starting with the black player), claiming one empty cell on the board per turn. Achieving one's own goal simultaneously leads to blocking the opponent's paths. An example board after the end of a game and a victory for the red and black players is presented in Fig. 1.



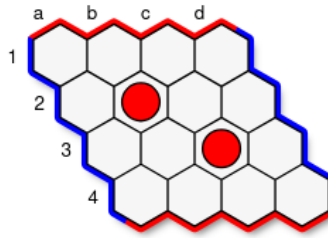
(a) Red player's victory: the chain connects the left and right edges.



(b) Black player's victory: the chain connects the top and bottom edges.

Rysunek 1: Example boards after the end of a match on a 9×9 board.

An important element of the game's mechanics, aimed at balancing the gameplay, is the *swap rule* (or *pie rule*). In the classic variant of the game, the first player has a significant advantage. The swap rule mitigates this problem by giving the second player a choice after the opponent's first move: they can either take over the played move as their own (swap) or decline this option and make a standard move [4]. This mechanism forces the starting player to make the most neutral and balanced opening move possible. An opening that is too strong might prompt the opponent to swap, while an opening that is too weak allows the opponent to gain an advantage without needing to use it.



Rysunek 2: Example of an interior template of the *bridge* type.

An important strategic aspect of the game are the so-called *interior templates* [3]. These are predefined configurations of one's own stones and empty cells that mathematically guarantee a connection, regardless of how the opponent responds. The simplest and most frequently used template of this type is the *bridge*. It is formed by two stones of the same color, between which there are exactly two empty cells sharing a common edge, as seen in Fig. 2. If the opponent occupies one of these empty cells, the player can always occupy the other, completing the connection despite the blocking attempt.

Understanding the topology of the board and templates is also crucial in defense. A fundamental mistake often made by beginners is blocking the opponent by placing stones directly in front of their chain. A much more effective strategy is blocking at a distance. Placing a blocking stone at a certain distance from the opponent's developing path forces them to change their plans and gives the defending player extra time to build a solid "wall" before the opponent's chain actually reaches it [1].

2 Approach 1: Heuristics

2.1 Opening phase and the swap rule

Regardless of the heuristic version, a common, fairly simple knowledge base regarding the first moves of the game was implemented:

- **Opening:** In 80% of cases, the bot chooses safe, balanced starting positions (to discourage the opponent from using the *swap rule*). In 20% of cases, it applies an aggressive strategy, occupying random cells in the strict center of the board, which, if not swapped, provide an advantage.
- **Swap rule:** If the bot makes the second move in the game, it analyzes the position of the opponent's stone. If it is located within a radius of two cells from the exact center of the board, the bot automatically activates the swap rule and takes over that stone.

2.2 Base Agent (Blind Heuristic Agent)

The first version of the bot (later referred to as **Blind Heuristic Agent**) was based on the simplest and most obvious assumption: occupy cells that form the shortest path between its own edges. For this purpose, a 0-1 Breadth-First Search (0-1 BFS) was used. The cost of passing through one's own stone was 0, through an empty cell 1, and cells occupied by the opponent were treated as inaccessible (infinite cost). The bot summed up the distance from a given empty cell to both of its edges, and in case of ties, it preferred cells located closer to the center. This bot correctly aimed for the goal; however, its main flaw was a complete lack of defensive play - it totally ignored the paths being built by the opponent. Moreover, its offensive strategy was very naive: the agent aimed to directly connect stones side by side, unable to utilize topologically advanced structures like bridges, which are much harder to block.

2.3 Improved Agent (Heuristic Agent)

To eliminate the weaknesses of the base version, another version of the bot was created (later referred to as **Heuristic Agent**). This agent calculates the shortest paths for both itself and the opponent. For every potential move, the bot generates an evaluation in the form of a tuple of values, based on which it makes the final decision. The key element of the evaluation is the *critical_score* - a value defined as the minimum of its own path length and the opponent's path length (increased by a small constant to slightly prefer its own attack over defense). Thanks to this adjustment, the bot can smoothly switch between pursuing its own goal and ruthlessly blocking the opponent when they are close to winning. It should be noted, however, that despite these improvements, this approach is still burdened with limitations resulting from the use of a classic shortest path algorithm. The bot's defense relies on directly cutting off the calculated route of the opponent, which means the agent cannot employ the strategically more effective distant blocking. Similarly, in the attack phase, the construction of its own chain is carried out in a naive manner, just like the **Blind Heuristic Agent**.

2.4 Attempts to use domain knowledge

During the development, attempts were also made to implement more complex heuristics consisting of a set of manually designed rules, such as prioritizing the creation of bridges or preemptive blocking at a distance. This approach was ultimately discarded because manually tuning the weights of the individual components of a move's evaluation for multiple rules proved to be problematic. Attempts to improve one aspect of the game usually broke the bot's behavior in other board situations, resulting in, for example, the bot scattering unconnected bridges across the board, forgetting the necessity to eventually close them and

form a coherent path. Ultimately, it turned out that the much simpler approach used in the **Heuristic Agent** easily defeated the manually tuned versions of the bot.

3 Approach 2: Optimization

The second approach is an extension of the idea of using domain knowledge from section 2.4. A set of board features was defined, and the task of finding the optimal weights for these features was entrusted to the **CMA-ES** (*Covariance Matrix Adaptation Evolution Strategy*) algorithm.

3.1 Individual representation and move evaluation

An individual in the CMA-ES population was represented as a vector of 12 real numbers. These values constituted the weights of an evaluation function, which assigned a value to all legal moves in each turn. The move’s score was calculated as a linear combination of the weight vector (from the individual) and the feature vector extracted from the current board state. The bot chose the move with the highest evaluation function score.

To evaluate a single move, 12 normalized environmental features were determined. A detailed summary and interpretation of the used features are presented in Table 1.

Feature name (in code)	Description
<code>my_path_score</code>	Normalized length of our player’s shortest path to the goal after making the move. A shorter path indicates a better position.
<code>opp_path_score</code>	Normalized length of the opponent’s shortest path to the goal.
<code>path_adv</code>	Difference between the length of our path and the opponent’s path - shows who has the advantage.
<code>center_score</code>	Determines how close the cell is to the center of the board. Cells closer to the center have a higher value.
<code>friend_neighbors</code>	Number of adjacent cells occupied by our stones.
<code>enemy_neighbors</code>	Number of adjacent cells occupied by the opponent’s stones.
<code>touched_components</code>	Number of our disjoint stone groups that will be connected by this move.
<code>component_size_after</code>	Size of the resulting stone group after the move in relation to the board size.
<code>bridge_support</code>	Number of new bridges created by the move.
<code>bridge_block</code>	Number of the opponent’s bridges blocked by this move.
<code>liberties</code>	Number of empty cells around the played move.
<code>swap_bias</code>	Feature used only on the second player’s first move to decide whether to use the swap rule.

Tabela 1: Features used to evaluate moves.

3.2 Fitness function and Curriculum Learning

The fitness function was the percentage of won games (win-rate). Given the nature of the game, evaluating individuals exclusively against a single, constant opponent leads to overfitting and learning a very narrow strategy, while introducing very strong opponents right away would lead to a situation where most individuals lose virtually everything (the sparse reward problem). To prevent this, the technique of gradually increasing the difficulty of opponents (*Curriculum Learning*) was applied. An essential element of the process was the *Hall of Fame* mechanism. It stored the best individuals from previous generations, which were used as opponents during evaluation. The optimization process was divided into three phases with increasing difficulty levels:

- **Initial phase (up to 30% of generations):** rapid rejection of the weakest individuals. The opponents were the random agent (60%) and the **Heuristic Agent** (40%).
- **Middle phase (30%-70% of generations):** the random agent’s share was reduced to 20%. The *Hall of Fame* (HoF) mechanism was introduced, with which 30% of games were played. The remaining 50% of games were played against the **Heuristic Agent**.
- **Final phase (from 70% of generations):** the random agent was completely removed. Individuals were evaluated against tougher opponents: **Heuristic Agent** (50% of games) and HoF agents (50%).

The final result of the process was the individual achieving the highest *win-rate* in the third, most difficult optimization phase.

Parameter	Value
Number of features (individual’s dimension)	12
Population size	24
Number of generations	120
Initial standard deviation (σ_0)	0.35
Hall of Fame size	10
Matches per individual evaluation	20
Fitness function	Win-rate
Optimization method	CMA-ES

Tabela 2: Parameters of the CMA-ES optimization process.

4 Approach 3: Reinforcement Learning

The most advanced approach involved applying Deep Reinforcement Learning methods. Because the number of available cells in Hex constantly decreases, the **PPO** algorithm with an action masking mechanism (*Maskable Proximal Policy Optimization*) from the *Stable Baselines3 (SB3)* library was utilized. This allows the agent to ignore illegal moves upfront (e.g., attempts to place a stone on an occupied cell), which saves time and significantly accelerates the learning process.

Additionally, to facilitate learning, a unification of perspective was applied, i.e., the agent always aims to connect the top with the bottom, which allows the network to focus on a single strategy instead of learning two directions simultaneously. In practice, this is implemented by swapping row and column coordinates (transposition), allowing the network to always analyze the situation vertically. After the model makes a decision, the coordinates are restored to their original layout so that the move lands in the right place on the actual board.

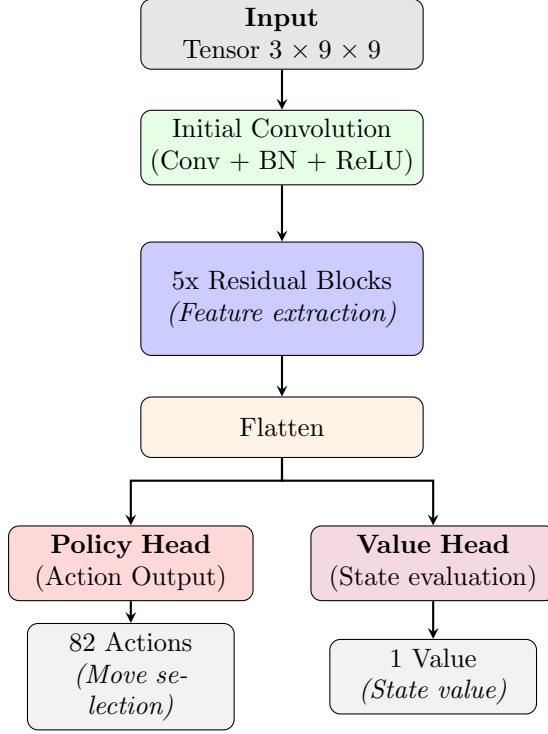
Another improvement is the use of the board’s 180° rotational symmetry as a form of data augmentation. For every board state, the network receives its ”twin” counterpart. This allows doubling the number of training examples without having to play new games.

4.1 State representation and network architecture

In order for the neural network to effectively analyze the situation on the board, the environment state was passed as a $3 \times 9 \times 9$ tensor. The three input channels encoded, respectively:

- positions of the player’s own stones,
- positions of the opponent’s stones,
- empty cells on the board.

A deep convolutional neural network based on the **ResNet** architecture was used for feature extraction. It consisted of an initial convolutional layer and five *residual blocks* with 128 filters each. After processing the data through the residual blocks and a flatten layer, the model transitions to the decision phase. The network’s output is a linear layer that generates values for 82 possible actions (81 board cells and the *swap rule* option). A detailed diagram is presented in Fig. 3.



Rysunek 3: Architecture of the PPO network

4.2 Phase 0: Imitation Learning (supervised learning)

In this phase, the model learned to mimic the moves of ready-made heuristic algorithms (*Behavioral Cloning*) on a dataset of 8000 played games. Cross-entropy was used as the loss function. With initial weights prepared this way, the actual reinforcement learning process could immediately focus on optimizing strategy and finding moves better than those resulting from simple heuristics, which significantly accelerated the model’s learning process.

4.3 League system and preventing forgetting

A key element in the implementation of the training environment was the created league module. Teaching an agent exclusively through matches against itself (*Self-Play*) carries the risk of catastrophic forgetting - the agent may learn a narrow strategy that defeats its current iteration, while forgetting the basic rules of defending against the simplest attacks. To counter this, the league module dynamically drew opponents for each game according to the following distribution:

- **50% - Heuristic agents:** They provided a stable benchmark, forcing the model to retain fundamental defensive and offensive capabilities against simple players.
- **25% - Historical models:** The agent competed against its older, previously saved versions. This mechanism guaranteed continuous progress and prevented strategic regression.
- **25% - Current model (Self-Play):** Competing against itself.

4.4 Training process (Curriculum Learning)

The actual training using the Maskable PPO algorithm was divided into two phases with an increasing level of difficulty, continuing the *Curriculum Learning* idea:

- **Critic Warmup:** For the first 200,000 steps, the weights of the pre-trained feature extractor and the actor’s decision head were frozen. The network only learned to evaluate the value of a given state. The opponent at this stage was strictly the heuristic agent. The goal of this procedure was to

teach the critic a proper evaluation of board states before unfreezing the actor, so that the actor’s initially random errors would not destroy the weights obtained in the BC process.

- **Main Training:** After the warmup, all model weights were unfrozen, and the full league system was launched. This phase lasted 3.5 million environment steps (which corresponds to roughly 170,000 played games, assuming an average of 20 agent moves per game). A linear learning rate schedule was applied, which allowed for precise fine-tuning of the model’s parameters in the final phases of training. This process could have been longer, but at this stage, the model had developed a strategy advanced enough to consistently beat a human player (the author of the project), while demonstrating effective blocking and the creation of non-obvious connections. It was decided that the obtained level of model intelligence was fully sufficient to achieve the project’s goals.

Parameter	Value
Learning rate	$5 \cdot 10^{-5}$ (linearly decreasing)
Entropy (<code>ent_coef</code>)	0.005
Clip range	0.15
Target KL	0.03
Number of rollout steps (<code>n_steps</code>)	512
Batch size	512

Tabela 3: Key hyperparameters for training the PPO model.

5 Comparison of results and analysis

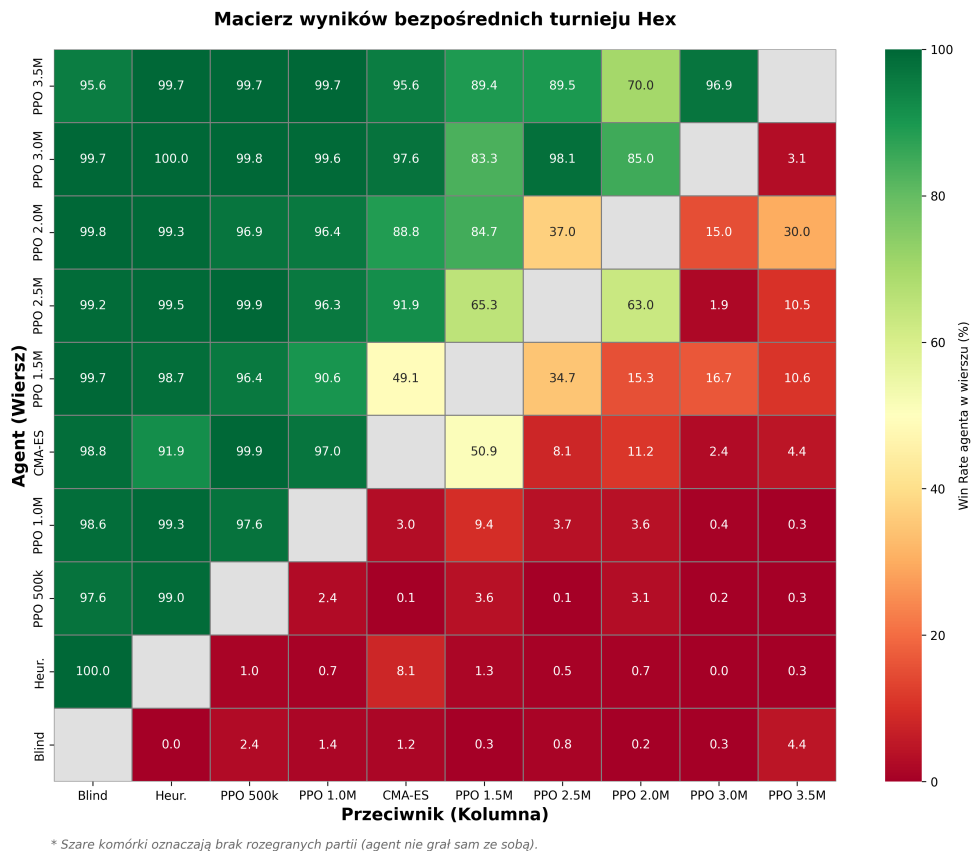
5.1 Bot effectiveness - tournament

To objectively evaluate the effectiveness of the implemented solutions, a tournament was held in which the agents played each other (in a round-robin system) for 1000 games - 500 as black and 500 as red. Aggregate statistics from these matches are summarized in Fig. 4, while the detailed results of head-to-head encounters are presented by the matrix in Fig. 5.

WYNIKI TURNIEJU					
Miejsce / Przeciwnik	Bot	Wygrane	Przegrane	Rozegrane	Win Rate (%)
▷ #1	PPO_3500000_steps	8361	639	9000	92.9%
▷ #2	PPO_3000000_steps	7662	1338	9000	85.13%
▷ #3	PPO_2000000_steps	6479	2521	9000	71.99%
▷ #4	PPO_2500000_steps	6275	2725	9000	69.72%
▷ #5	PPO_1500000_steps	5118	3882	9000	56.67%
▷ #6	CMA-ES	4646	4354	9000	51.62%
▷ #7	PPO_1000000_steps	3159	5841	9000	35.1%
▷ #8	PPO_500000_steps	2064	6936	9000	22.93%
▷ #9	Heuristic Agent	1126	7874	9000	12.51%
▷ #10	Blind Heuristic Agent	110	8890	9000	1.22%

Rysunek 4: Results table of matches between bots.

The PPO 3.5M model (trained for 3.5 million steps) turned out to be the undisputed winner of the tournament, achieving a total win rate of 92.9%. Analyzing the general breakdown, one can clearly see a positive correlation between the length of the training process and the final effectiveness of models based on the PPO algorithm. Usually, each successive version of the agent performed better and better in the tournament, but in the detailed round-robin match statistics, some deviations can be noticed. The PPO 2.0M model ranked higher in the overall standings (third place with a 71.99% win rate) than its longer-trained counterpart PPO 2.5M. Looking at the head-to-head matches with the master 3.5M version in the matrix, one can see that the PPO 2.0M agent loses to it in 70% of games, whereas the newer, theoretically stronger 3.0M iteration succumbs to the master in a whopping 96.9% of cases. Such deviations can be explained by the specifics of reinforcement learning and the league module in the main training. During the training of the 3.5M version, the 2.0M agent might simply have been drawn less frequently as an opponent from the pool of historical models. Furthermore, the 2.0M iteration might have developed a very



Rysunek 5: Matrix of head-to-head results in the Hex bots tournament. Values in the cells represent the win percentage of the agent in a given row against the agent in a given column.

unusual playstyle at that time, to which the final model didn't manage to fully build immunity. This may also be a good explanation as to why the 2.5M agent achieved a worse win-rate in the overall ranking. It's possible that it was in a "transitional phase", in which it departed from the effective but specific strategy of the 2.0M version, and was only just evolving toward a new, ultimately optimal concept that began to stabilize in the 3.0M and 3.5M versions.

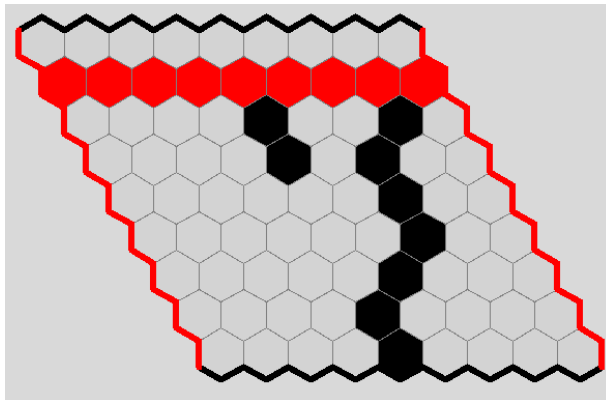
When analyzing the effectiveness of the remaining implemented approaches, one must appreciate the agent created using the optimization method. It was able to compete on equal terms with the reinforcement learning model halfway through its training process. CMA-ES developed a minimal head-to-head advantage, winning 50.9% of games played against the PPO agent trained for 1.5 million steps, while simultaneously placing just behind it in the overall ranking, in sixth place with a 51.62% win rate.

On the opposite extreme, as expected, were the heuristic bots based on the shortest-path algorithm. Both Heuristic Agent and Blind Heuristic Agent recorded low total win rates in the tournament - amounting to 12.51% and a mere 1.22%, respectively. However, an in-depth analysis of the head-to-head results matrix shows that the Heuristic Agent owes its 12.51% win rate almost entirely to its absolute domination over the base version - winning exactly 100% of games against the Blind Heuristic Agent. But an even more interesting phenomenon that can be observed is that the weakest in the set, the Blind Heuristic Agent, recorded its highest win percentage (4.4%) against the absolute tournament winner, the PPO 3.5M model. This phenomenon is absent in the earlier phases of learning - weaker models, such as PPO 2.5M or 3.0M, handled these naive attacks almost flawlessly (suffering losses in only 0.2% - 0.3% of games). This seemingly illogical anomaly can be explained by the phenomenon of "overfitting" to advanced opponents. The PPO agent in the final training phase was optimized mainly to defend against complex and well-thought-out attacks. It can be assumed that it began to slightly disregard the absurdly simple and naive attacks of the weakest bot, which is why it unexpectedly lost in as many as 44 out of 1000 clashes.

5.2 Behavioral analysis

5.2.1 Heuristic bots

An example of gameplay presented in Figure 6 shows the lack of usefulness of the base agent (**Blind Heuristic Agent**) in practical play against a human player due to its fundamental flaws in defensive play and extreme predictability. In the context of this project, it serves purely as a simple benchmark (*baseline*).



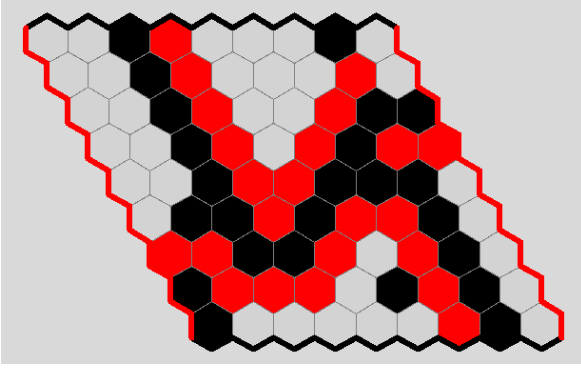
Rysunek 6: An example of naive behavior by the Blind Heuristic Agent (black stones). The human player (red stones) easily blocked the bot’s path with a simple horizontal line, thus winning the game.

In turn, the heuristic bot (**Heuristic Agent**) serves as a good representation of a beginner human player. Although, unlike the base version, it can react to the opponent’s moves, it does so in a naive way, basing its defense on directly cutting off the path. This algorithm does not show a deeper understanding of board topology. It cannot apply blocking at a distance or intentionally build advanced, disruption-resistant structures. Despite these limitations, its behavior is coherent and goal-oriented enough that it works perfectly as a basic low-difficulty opponent, providing an appropriate training environment for people who are just learning how to play. Typical behavior of this agent is illustrated by the two sample matches presented in Fig. 7. It is clearly visible that by using a naive approach based on direct blocking (left side), the player meets significant resistance from the algorithm, and the match becomes fierce and dense with stones. On the other hand, the moment a human starts playing more thoughtfully, using topological knowledge and interior templates such as bridges (right side), the bot’s simple heuristic proves completely helpless in trying to block the rival in any way. Figure 8 presents the opening principles and swap rules implemented in the heuristics.

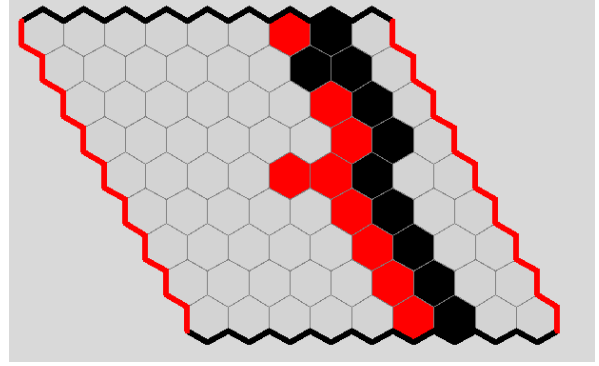
5.2.2 Bot created using the optimization algorithm

An analysis of the weight vector optimized by the evolutionary CMA-ES algorithm (presented in Table 4) reveals a very specific and unexpected gameplay strategy that significantly deviates from the classic approach and confirms the findings from practical matches against this agent. First and foremost, the model exhibits an extremely defensive mindset. The weight of the feature evaluating the opponent’s path ($\text{opp_path_score} = 24.64$) is three times higher than the weight assigned to its own path ($\text{my_path_score} = 8.06$). This means that the bot prioritizes blocking the opponent above all else, and treats forming its own connection as a secondary goal. The biggest surprise, however, is the strongly negative weight assigned to the center_score feature (-5.51). In classic Hex strategy, occupying the center of the board is considered crucial for achieving victory. The CMA-ES algorithm “decided” the opposite - it actively avoids the center cells, preferring to play near the edges. This is corroborated by the generated evaluation map at opening (Figure 9a), on which central cells hold the lowest value, while the corners of the board are evaluated the highest. This also explains the bot’s surprising behavior regarding the swap rule. As seen in Figure 9b, the CMA-ES agent never utilizes this option (0% across all cells).

However, its high effectiveness in games against other bots, confirmed in the tournament, does not translate to confrontations with a human. By training against a limited pool of artificial opponents, the

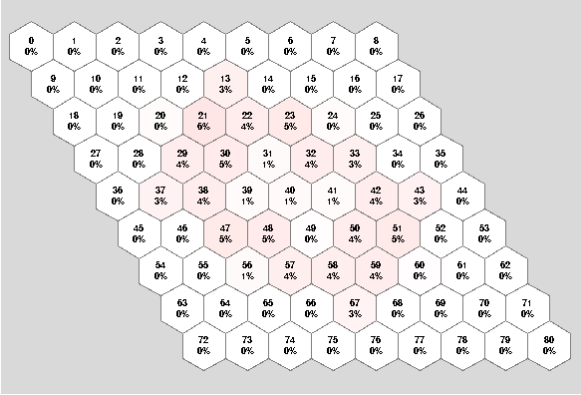


(a) Naive strategy: no bridges and direct blocking.

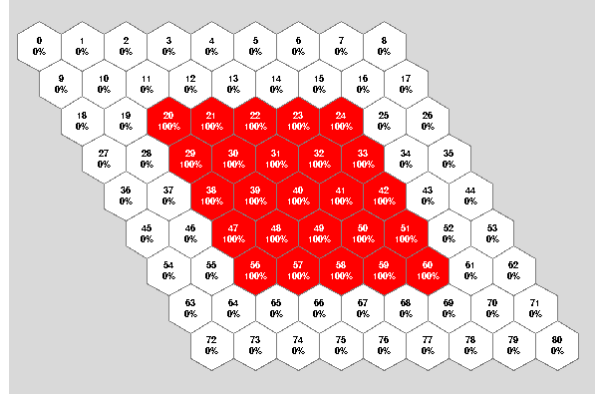


(b) Advanced strategy: an easy win thanks to bridges and distant blocking.

Rysunek 7: Comparison of two victorious games by a human player (black color) against the **Heuristic Agent** (red color). The figure on the left shows a game played naively without using templates, which led to a fierce fight and a densely filled board. The figure on the right shows a fully controlled victory achieved by consciously employing bridges.



(a) Agent's opening distribution

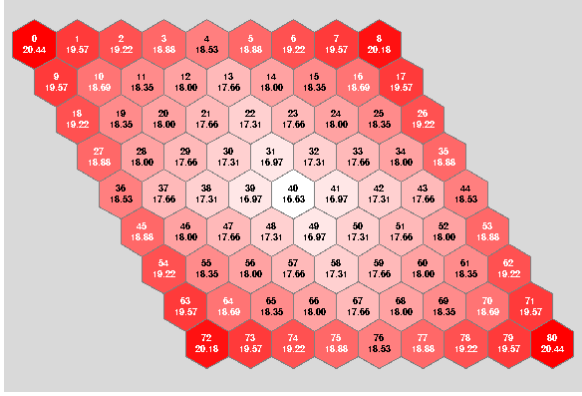


(b) Swap rule decisions

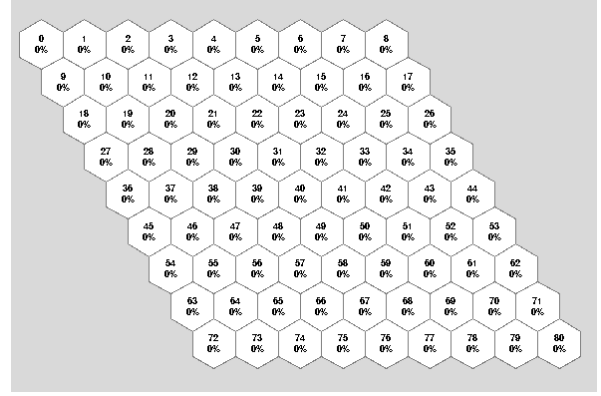
Rysunek 8: Analysis of the opening phase and swap rule for the Heuristic Agent.

Feature name	Weight value
my_path_score	8.0644
opp_path_score	24.6421
path_adv	-3.6588
center_score	-5.5106
friend_neighbors	-4.4368
enemy_neighbors	1.0500
touched_components	6.7797
component_size_after	8.3478
bridge_support	1.9727
bridge_block	-4.0666
liberties	-1.5874
swap_bias	0.1151

Tabela 4: Weight values of individual features optimized by the CMA-ES algorithm.



(a) Cell evaluations during the opening



(b) Swap rule decisions

Rysunek 9: Analysis of the opening phase and swap rule for the CMA-ES agent.

bot found a specific local optimum. The extremely defensive play and building its own path near the edges successfully exploited the logic gaps in weaker agents; yet, in direct confrontation with a human, this rigid and highly predictable strategy proves completely ineffective.

5.2.3 Bot created using reinforcement learning

The learning process of the PPO model demonstrates how the agent gradually evolves, discarding rigid rules in favor of a deep understanding of board topology and advanced strategy. Observations of opening preferences (Figure 11) and swap decisions (Figure 10) at different stages of training, supported by direct matches between the bots and the author, allow drawing several key conclusions:

Early learning phase ($\leq 1.5M$ steps)

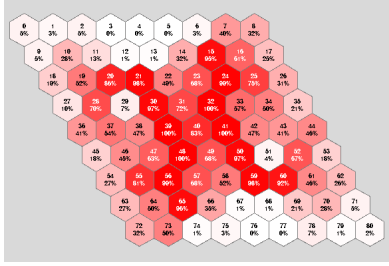
Early models still heavily rely on the knowledge gathered from the Imitation Learning phase. This is visible in the broad, almost sprawling acceptance area for the swap rule (Figure 10a and 10b). Although this model can occasionally form bridges, its understanding of these structures is superficial - it often fails to close them correctly or properly defend against similar constructions by the opponent. From the perspective of an advanced human player, models from this phase are relatively easy to beat.

Transitional phase and shift (2.0M – 2.5M steps)

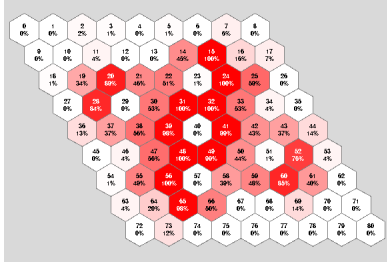
The middle training phase reveals a phenomenon of temporary destabilization of the agent’s policy. The PPO 2.5M version features the most scattered opening decisions (Figure 11d), which, combined with a worse tournament result (a drop from 71.99% to 69.72% win-rate), suggests a transitional period in learning. At this stage, the agent has already discarded the simple, safe heuristics acquired in phase 0, but has not yet fully developed stable defensive mechanisms. In direct play, this manifests as slight “confusion” of the 2.5M bot - while it attempts to create advanced structures, it often fails to defend effectively at a distance against attacks used by humans.

Final phase (3.0M – 3.5M steps)

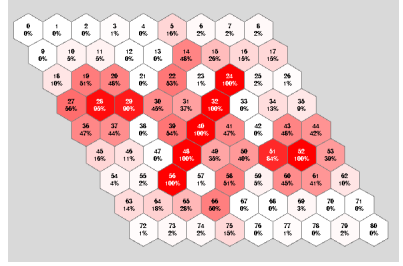
The final models showcase a strategic level that can successfully compete with, and in many cases completely dominate, a human player. As seen in Figures 10e and 10f, the 3.0M and 3.5M models have developed a swap execution zone around the board’s diagonal. In head-to-head encounters, PPO 3.0M can fluently utilize distant blocking and structure building, making matches with the human player (the project’s author) highly balanced. However, the 3.5M version displays the highest level. This model not only effectively blocks human progress, but does so in such a way that every defensive stone simultaneously becomes a part of its own offensive game. This often leads to situations where the human player, focused on trying to break a block, suddenly realizes (often 3-4 moves before the end) that the board state is already mathematically lost, and every prior inaccuracy was ruthlessly exploited by the network.



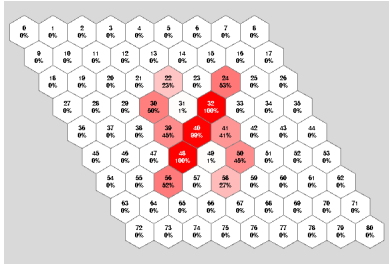
(a) PPO 1.0M



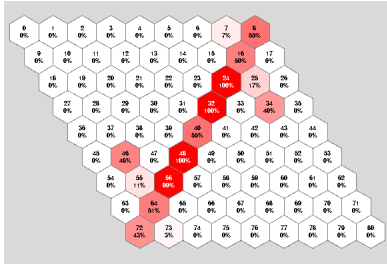
(b) PPO 1.5M



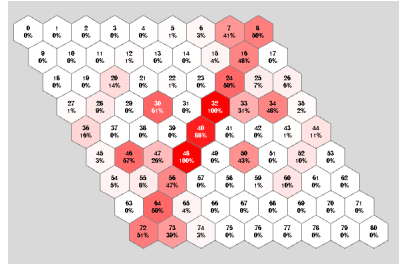
(c) PPO 2.0M



(d) PPO 2.5M

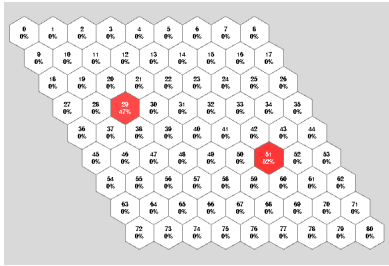


(e) PPO 3.0M

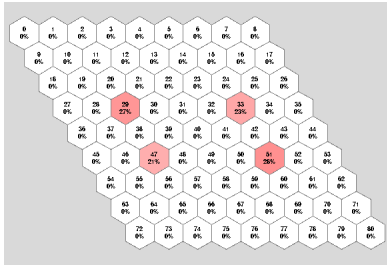


(f) PPO 3.5M

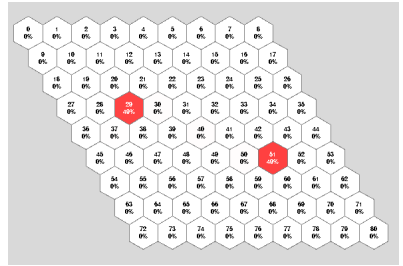
Rysunek 10: Comparison of swap rule operation depending on the number of learning steps for the PPO model.



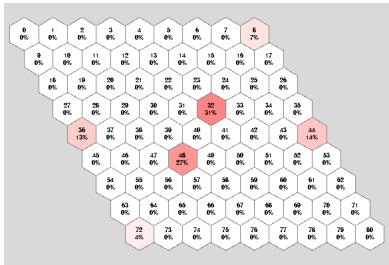
(a) PPO 1.0M



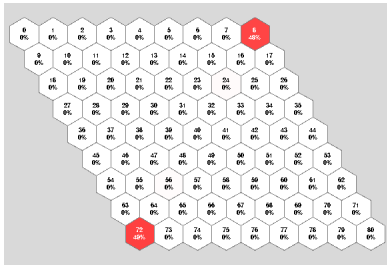
(b) PPO 1.5M



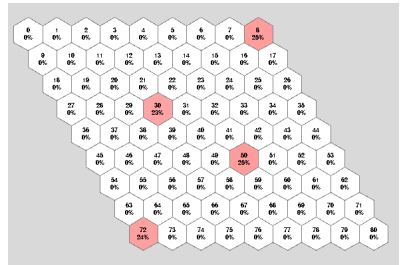
(c) PPO 2.0M



(d) PPO 2.5M



(e) PPO 3.0M



(f) PPO 3.5M

Rysunek 11: Comparison of opening moves depending on the number of learning steps for the PPO model.

5.3 Comparison of workload and efficiency of approaches

The only cost in the case of heuristic agents was the short time spent thinking through and implementing their logic. However, they achieve the poorest results and behave in a highly predictable manner for the opponent. In the CMA-ES approach, the workload was high due to the necessity of manually designing functions extracting board features, but the weight optimization process itself was brief (approx. 40 minutes).

In contrast, the method utilizing reinforcement learning required the greatest amount of work (network and league architecture, Imitation Learning phase) and a multi-hour training process (approx. 5–6 hours for 3.5 million steps). Despite a significant consumption of resources, only the approach using PPO allowed for creating a high-level agent capable of autonomously discovering complex defensive and offensive strategies that go beyond the rigid logic defined in the source code.

Literatura

- [1] HexWiki. Basic Strategy Guide. URL: [https://www.hexwiki.net/index.php/Basic_\(strategy_guide\)](https://www.hexwiki.net/index.php/Basic_(strategy_guide)).
- [2] HexWiki. Draw. URL: <https://www.hexwiki.net/index.php/Draw>.
- [3] HexWiki. Interior template. URL: https://www.hexwiki.net/index.php/Interior_template.
- [4] HexWiki. Swap rule. URL: https://www.hexwiki.net/index.php/Swap_rule.